

Assignment: Hospital Placement with Hill Climbing

Optimization and Search

Objective

In this assignment, you will implement and analyze the **Hill Climbing algorithm** to solve a simplified **hospital placement problem**. The task is to place hospitals on a grid so that the sum of distances from houses to their nearest hospital is minimized. You will use the provided Python template (`hospitals_exercise.py`), which defines the problem setup and leaves key parts of the algorithm as TODOs.

Instructions

1) Understand the Problem Setup

- You are given a 2D grid with houses placed at certain coordinates.
- Your goal is to place a fixed number of hospitals on empty cells.
- The objective function is the *total Manhattan distance* between each house and its nearest hospital.

2) Complete the Missing Functions

- `get_cost(hospitals)`: Implement the cost function. For each house, compute the Manhattan distance to the closest hospital and return the sum over all houses.
- `get_neighbors(row, col)`: Implement neighbor generation. Given a hospital location (row, col) , return all valid neighboring cells (up, down, left, right) that are not occupied by houses or other hospitals.
- `hill_climb(...)`: Implement the Hill Climbing algorithm:
 - (a) Start with a random placement of hospitals.
 - (b) Iteratively try moving one hospital to one of its neighbors.
 - (c) If the move improves the cost, accept it.
 - (d) Stop when no improvement is possible or when a maximum number of steps is reached.

3) Run Your Algorithm

- Use the optional visualization (`output_image`) to generate images of your steps and the final configuration.

4) Random Restart

- After completing the basic Hill Climbing, implement *Random Restart Hill Climbing*.
- Run multiple independent hill climbs, each starting from a random initial placement, and keep the best solution found.
- Compare the performance (final cost) with and without random restarts.

5) (Optional) Simulated Annealing Extension

- If you have extra time, extend your code with a function `simulated_annealing(...)`.
- Decrease T gradually after each step (e.g., $T \leftarrow \alpha T$ with $\alpha \approx 0.98$).
- Stop when the maximum number of steps is reached or when T becomes very small.
- Compare the behavior of Simulated Annealing with Hill Climbing: does it sometimes find better solutions by escaping local optima?
- **Note:** This part is entirely optional and will *not* affect your grade. It is included for those who want to experiment further.

Questions to Answer

Alongside with your code, provide the extended answers to the following questions. You can hand them in in PDF format or similar.

- 1) **Local search versus search** How is local search different than other search problems encountered in the course (e.g., solving a maze with DFS or BFS)?
- 2) **Local vs. Global Optima** What is the difference between a local optimum and a global optimum? Provide an example from your experiments where Hill Climbing got stuck in a local optimum.
- 3) **Effect of Random Restart** What is random restart and how does it increase the probability of finding the global optimum?
- 4) **Greedy Nature of Hill Climbing** Why can a greedy (steepest-ascent) strategy fail in this problem? Would allowing occasional “worse” moves (as in Simulated Annealing) help? Why or why not? Describe what changes you would have to apply to the algorithm to implement Simulated Annealing.

Deliverables

- Completed Python code (`hospitals_exercise.py` with all TODOs filled in).
- A short report (1–2 pages) including:
 - Answers to the questions above.
 - Example outputs (final placements, costs).
 - Screenshots of your visualization (optional).

Grading

Your assignment will be evaluated as either passed or not passed. If you do not pass the assignment, you will have the chance to resubmit after receiving feedback. The conditions for passing are listed in the rubrics below:

Component	Pass	Not Pass
Implementation (60%)	Code runs without fatal errors. Hill climbing logic is clearly implemented (neighbors, evaluation, updating). Produces reasonable results on the given problem (may not always find the global optimum, but works as expected).	Code does not run, or is missing. Hill climbing is not implemented (random guessing, brute force, or unrelated algorithm). Output is nonsensical or missing.
Theory Questions (40%)	Answers demonstrate understanding of key ideas: local vs. global optima, random restart, difference from simulated annealing. Explanations are mostly correct and coherent (minor errors are acceptable).	Answers show fundamental misconceptions. Key questions left unanswered. Explanations are incoherent or irrelevant.

Pass requirement: Both parts must be at “Pass” level (adequate implementation *and* adequate theory). Minor mistakes are fine, but there must be evidence of effort and basic competence.